

Performance & Scalability Guide

Enterprise Resolution Intelligence System API v3.5

1. Latency Expectations & SLAs

Operation	P50	P95	P99	Notes
GET /tickets (list)	50ms	150ms	300ms	Cacheable, pagination required
GET /tickets/:id	30ms	80ms	150ms	Single record, cache-friendly
POST /tickets	100ms	250ms	500ms	Writes to DB, invalidates cache
POST /analytics	500ms	1.5s	3s	Heavy computation, async preferred
POST /search	200ms	600ms	1.2s	Full-text search, index-dependent

Latency Optimization Tips:

- Use pagination (max 100 items per page) to reduce response size
- Implement field filtering with ?fields=id,status,priority to reduce payload
- Leverage ETags for conditional requests (If-None-Match header)
- Choose nearest region endpoint to minimize network latency

2. Throughput & Capacity Limits

Tier	Req/Second	Concurrent Connections	Burst Capacity	Payload Limit
Free	5	10	20/10s	1 MB
Basic	50	100	100/10s	5 MB
Professional	200	500	400/10s	10 MB
Enterprise	1,000+	5,000+	Custom	50 MB

3. Caching Strategy & Mechanisms



Latency: ~10ms (cache) vs ~100ms (DB)
Cache Hit Ratio Target: >90%

3.1 Cache Control Headers

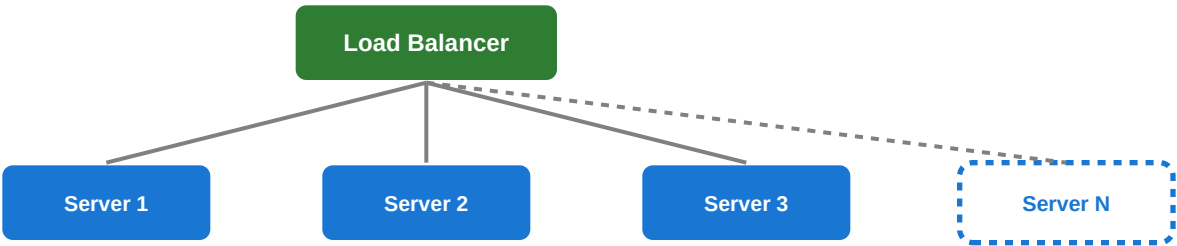
Header	Value	Use Case
--------	-------	----------

Cache-Control	public, max-age=300	Cache for 5 minutes (read-heavy data)
Cache-Control	private, max-age=60	User-specific data, 1-minute cache
Cache-Control	no-cache	Always revalidate with server (ETag)
ETag	"abc123xyz"	Resource version for conditional requests

3.2 Client-Side Caching Example

```
# Python with requests-cache
import requests_cache
session = requests_cache.CachedSession('api_cache', expire_after=300)
response = session.get('https://api.example.com/v3/tickets')
```

4. Scaling Best Practices



Horizontal Scaling: Add more servers to handle load

4.1 Horizontal vs Vertical Scaling

Aspect	Horizontal Scaling	Vertical Scaling
Definition	Add more servers/instances	Increase CPU/RAM of existing server
Cost	Linear cost increase	Exponential cost at high tier
Availability	High (redundancy)	Single point of failure
Complexity	Load balancing required	Simple architecture
Limit	Nearly unlimited	Hardware ceiling
Recommended	✓ Production systems	Development/testing

4.2 Connection Pooling

Reuse persistent connections to reduce overhead:

```
# Python example
from requests.adapters import HTTPAdapter
session = requests.Session()
adapter = HTTPAdapter(pool_connections=10, pool_maxsize=20, max_retries=3)
session.mount('https://', adapter)
```

- **pool_connections**: Number of connection pools to cache
- **pool_maxsize**: Max connections per pool
- **max_retries**: Retry failed requests automatically

4.3 Async/Batch Operations

- Use /v3/batch endpoint to create multiple tickets in single request (up to 100)
- Leverage webhooks for async notifications instead of polling
- Implement request queuing for non-critical operations
- Use streaming responses for large datasets (Transfer-Encoding: chunked)

5. Regional Deployment & Optimization

Region	Endpoint	Avg Latency (Global)	Features
US East	us-east.api.example.com	50-150ms	Primary, full feature set

US West	us-west.api.example.com	40-140ms	Full redundancy
EU Central	eu.api.example.com	60-180ms	GDPR compliant
Asia Pacific	apac.api.example.com	80-250ms	Regional isolation
Global	api.example.com	Auto-route	GeoDNS routing

Regional Selection Strategy:

- Auto-routing: Use global endpoint for automatic routing to nearest region
- Manual selection: Pin to specific region for data residency requirements
- Multi-region: Deploy in multiple regions for disaster recovery (Enterprise)
- Latency test: Use /v3/ping endpoint to measure round-trip time

6. Performance Monitoring Metrics

Metric	Threshold	Action
Response Time (P95)	> 500ms	Enable caching, optimize queries
Error Rate	> 1%	Check logs, review recent changes
Cache Hit Ratio	< 85%	Increase TTL, review cache strategy
Rate Limit Usage	> 80%	Implement backoff, upgrade tier
Connection Pool Exhaustion	Frequent	Increase pool size, check leaks
Timeout Rate	> 0.1%	Optimize queries, increase timeout

7. Quick Optimization Checklist

✓	Optimization	Impact
■	Enable response compression (gzip)	50-70% bandwidth reduction
■	Implement client-side caching	80-90% reduction in API calls
■	Use connection pooling	30-40% latency improvement
■	Paginate large result sets	10x faster response times
■	Select nearest region endpoint	50-200ms latency reduction
■	Use batch endpoints for bulk operations	90% reduction in requests
■	Implement conditional requests (ETags)	Zero payload on 304 responses

Performance Support

Performance Dashboard: <https://status.example.com/performance>
Optimization Guide: <https://docs.example.com/optimization>
Architecture Consulting (Enterprise): architecture@example.com